SupplyFlow

Product Requirements Document (PRD)

Course: CS6983 – Project 2: Full-Stack Application
Project Type: Pair Project
Team Members: Rohan Kumar and Partner (TBD)
Date: October 24, 2026
Version: 1.1

---

1. Project Overview

Our project, SupplyFlow, is a full-stack web application designed to help companies manage their supply chain operations through a centralized platform.

Many businesses rely on spreadsheets, emails, and disconnected systems to manage vendors and purchase orders. This can make it difficult to track purchasing activities, maintain accurate vendor information, and monitor deliveries.

Our application will provide employees with a single platform to manage vendors, create and modify purchase orders, track delivery statuses, and view supply chain metrics through an interactive dashboard.

2. Project Objectives

Our goal is to build a user-friendly supply chain management application that allows companies to:

- Add, edit, deactivate, and search vendors.
- Create, update, approve, and cancel purchase orders.
- Assign purchase orders to vendors.
- Track order progress from creation to delivery.
- Monitor purchasing activity through an analytics dashboard.
- Securely access the application through authentication and role-based permissions.

3. Target Users

Administrator

Administrators will have permission to:

- Add, edit, and deactivate vendors.
- View all purchase orders.
- Approve pending purchase orders.
- Update order delivery statuses.
- Access the analytics dashboard.

Employee

Employees will have permission to:

- View and search vendors.
- Create purchase orders.
- Edit or cancel their own draft orders.
- Submit orders for approval.
- View order statuses and dashboard metrics.

The application will use two predefined demo accounts to simplify authentication testing.

4. Core Features and User Stories

Feature 1: User Authentication

US-01: Secure Login

As an employee, I want to securely log in so that I can access the supply chain platform.

Acceptance Criteria:

- Users can log in with an email and password.
- The backend authenticates credentials and issues a JWT.
- Invalid credentials display an error message.
- Protected pages require authentication.
- Users can log out.

US-02: Role-Based Access Control

As an administrator, I want users to have different permissions so that sensitive actions remain restricted.

Acceptance Criteria:

- The system supports Admin and Employee roles.
- Only administrators can create or edit vendors.
- Only administrators can approve purchase orders.
- Employees can create and manage their own draft orders.
- Unauthorized API requests return HTTP 403.
- The frontend hides restricted controls.

Feature 2: Vendor Management

US-03: Add Vendor

As an administrator, I want to add vendors so that employees can assign purchase orders to them.

Acceptance Criteria:

- Administrators can open an Add Vendor form.
- Users can enter vendor name, email, phone, and status.
- Required fields are validated.
- Vendor information is saved in PostgreSQL.
- The vendor appears in the vendor table.

US-04: Edit and Deactivate Vendors

As an administrator, I want to update vendor information so that supplier records remain accurate.

Acceptance Criteria:

- Administrators can edit vendor details.
- Administrators can deactivate vendors.
- Inactive vendors cannot receive new purchase orders.
- Existing purchase orders retain their vendor references.

US-05: Search and View Vendors

As an employee, I want to search vendors so that I can quickly find supplier information.

Acceptance Criteria:

- Users can view a vendor table.
- Users can search vendors by name.
- Users can filter vendors by status.
- Users can view vendor details.

Feature 3: Purchase Order Management

US-06: Create Purchase Orders

As an employee, I want to create purchase orders so that I can request products from vendors.

Acceptance Criteria:

- Employees can select an active vendor.
- Employees can add products, quantities, and prices.
- Employees can select an expected delivery date.
- Order totals are calculated automatically.
- Orders are saved with Draft status.

US-07: Edit or Cancel Orders

As an employee, I want to modify orders so that I can correct purchasing requests.

Acceptance Criteria:

- Employees can edit their own Draft orders.
- Employees can change quantities and prices.
- Order totals recalculate automatically.
- Eligible orders can be canceled.
- Delivered orders cannot be modified.

US-08: Approve Purchase Orders

As an administrator, I want to approve purchase orders so that purchasing requests are reviewed.

Acceptance Criteria:

- Employees can submit Draft orders for approval.
- Submitted orders change to Pending status.
- Administrators can approve Pending orders.
- Approved orders change to Approved status.
- Employees cannot approve orders.

Feature 4: Order Tracking

US-09: Track Purchase Orders

As an employee, I want to track orders so that I can monitor delivery progress.

Acceptance Criteria:

- Users can view order statuses.
- Administrators can update approved orders to Shipped and Delivered.

- Users can filter orders by status.
- Invalid status transitions are rejected.
- Changes appear without reloading the entire application.

Feature 5: Analytics Dashboard

US-10: View Dashboard Metrics

As an employee or administrator, I want to view supply chain metrics so that I can monitor purchasing activity.

Acceptance Criteria:

- Dashboard displays total orders.
- Dashboard displays active vendors.
- Dashboard displays pending orders.
- Dashboard displays total order spending.
- Dashboard highlights overdue deliveries.
- Metrics refresh when relevant order or vendor data changes.

5. Application Screens

Our application will include four main screens.

Login Page: Email and password fields, login button, and error messages.

Dashboard: Summary cards showing total orders, active vendors, pending orders, total spending, and recent order activity.

Vendor Management: Vendor table, search and filters, Add Vendor form, and Edit Vendor form.

Purchase Order Management: Purchase order table, Create Order form, order details, and status controls.

UI/UX Requirements

- Professional enterprise-style dashboard
- Consistent sidebar navigation
- Responsive layouts
- Accessible forms and buttons
- Clear validation and error messages
- Loading and empty states

Mockup References: To be added after creating the initial designs in Claude Artifacts or Figma.

6. Technology Stack

Component	Technology
Frontend	React + Tailwind CSS
Backend	Node.js + Express
Database	PostgreSQL
Authentication	JWT
API Documentation	Swagger/OpenAPI
Unit Testing	Vitest
Integration Testing	Supertest
E2E Testing	Playwright
CI/CD	GitHub Actions
Deployment	Vercel + Render
AI Development	Antigravity + AGENTS.md

7. Database Design

The application will use four primary tables.

Users

- id (Primary Key)
- name
- email
- password_hash
- role

Vendors

- id (Primary Key)
- name
- email
- phone
- status

Orders

- id (Primary Key)
- vendor_id (Foreign Key)
- created_by (Foreign Key)
- status
- expected_delivery_date
- total_amount

Order_Items

- id (Primary Key)
- order_id (Foreign Key)
- product_name
- quantity
- unit_price

Relationships:

- One vendor can have multiple orders.
- One user can create multiple orders.
- One order can contain multiple order items.

8. API Requirements

The backend will provide RESTful API endpoints.

Method	Endpoint	Description
POST	/api/auth/login	Authenticate user
GET	/api/auth/me	Retrieve current user
GET	/api/vendors	Retrieve vendors
POST	/api/vendors	Create vendor
PUT	/api/vendors/	Update vendor
GET	/api/orders	Retrieve orders
POST	/api/orders	Create order
PUT	/api/orders/	Update order
PATCH	/api/orders//status	Update order status

GET	/api/dashboard	Retrieve dashboard metrics

All endpoints will be documented using Swagger/OpenAPI.

9. Testing Strategy

We will follow test-driven development using the red-green-refactor approach.

Unit Testing: Input validation, order total calculations, role permissions, and status transition logic.

Integration Testing: Authentication, vendor operations, purchase order operations, and database interactions.

End-to-End Testing: Login, vendor creation, purchase order creation, order approval, and dashboard updates.

Failing tests will be committed before implementation to demonstrate TDD.

10. Agile Development Plan

Sprint 1 — October 24–31

- Configure React, Express, and PostgreSQL.
- Implement JWT authentication.
- Implement Admin/Employee roles.
- Build vendor management.
- Build purchase order creation.
- Create initial unit and integration tests.

Sprint 2 — November 1–12

- Complete purchase order management.
- Implement approval and order tracking.
- Build the analytics dashboard.
- Add E2E tests.
- Configure CI/CD and security scanning.
- Deploy the application.
- Complete documentation and demo video.

Both partners will contribute code, review pull requests, participate in standups, and document sprint retrospectives.

11. AI-Assisted Development

We will use Antigravity as our primary AI coding environment.

Our shared `AGENTS.md` file will document:

- Technology stack and architecture
- Coding conventions
- Naming standards
- Testing requirements
- PRD and mockup references
- GitHub branch and pull request workflow
- Security and accessibility practices

We will review and test AI-generated code before merging it.

12. CI/CD and Deployment

GitHub Actions will run linting, automated tests, and security checks on pull requests.

The application will use deployment previews and automated production deployment.

Environment variables will be used to manage sensitive credentials.

13. Out of Scope

The initial application will not include:

- AI demand forecasting
- Payment processing
- External ERP integrations
- Warehouse management
- GPS shipment tracking
- Automated vendor email notifications

14. Expected Deliverables

- Deployed full-stack application
- GitHub repository with contributions from both partners
- Two Agile sprints and GitHub Project board
- Shared AGENTS.md rules file
- Automated tests and CI/CD
- API documentation

- Architecture and database diagrams
- README and deployment guide
- 10-minute demo video
- Partner contribution log
- Individual reflections

15. Conclusion

SupplyFlow will provide a centralized system for managing vendors, purchase orders, and delivery activity.

Our project will demonstrate how full-stack development, database integration, authentication, automated testing, and AI-assisted engineering can be combined to create a practical business application.